

Guia mental: Bot de presupuesto por WhatsApp

Documento de referencia rapida del modulo Presupuesto Whatsapp. Es independiente de FacturAI (facturas) y del portal Openix (facturacion de clientes a Openix).

1. Que problema resuelve

Un cliente escribe por WhatsApp, elige servicios de un catalogo numerado (1, 2, 3...) y al escribir LISTO recibe un presupuesto con IVA. La empresa configura tarifas sin tocar codigo.

2. Tres productos distintos en el repositorio

- FacturAI (Automatizacion Facturas/): gestion de facturas. No interviene aqui.
- Portal Openix (Pagina web/portal.html): lo que el cliente paga a Openix.
- Presupuesto WhatsApp: este modulo + API chatbot puerto 8020.

3. Arquitectura en capas

Capa A - Pantallas HTML:

- index.html: menu de enlaces.
- configuracion.html: subir datos y publicar el bot.
- simulador.html: probar chat sin WhatsApp real.
- arquitectura.html: diagrama Meta vs BuilderBot.

Capa B - API FastAPI (chatbot-backend, puerto 8020):

- routes_tenant_config.py: catalogo, documentos, publicar.
- routes_whatsapp.py: webhook Meta, simular, health.
- presupuesto_service.py: motor por empresa (tenant).
- whatsapp_bot.py: envio a WhatsApp Cloud API.

Capa C - Motor Python:

- motor_presupuesto.py: conversacion (menu, LISTO, IVA).
- almacen_sesiones.py: estado por telefono (wa_id).
- generador_config.py: subidas -> catalogo.json + conocimiento.md.

4. Archivos y carpetas clave

catalogo_ejemplo.json

Demo. Fallback si no hay tenant publicado.

data/tenants/{id}/catalogo.json

Catalogo real. Fuente de precios del bot.

data/tenants/{id}/conocimiento.md

Contexto de textos subidos.

data/tenants/{id}/uploads/

Archivos originales subidos.

start-presupuesto-whatsapp.command

Arranque API en macOS.

.env en chatbot-backend

Tokens Meta y PRESUPUESTO_TENANT_DEFAULT.

5. Flujo completo

- Arrancar API (uvicorn puerto 8020).
- Abrir configuracion.html.
- Elegir ID empresa (ej. mi-clinica).
- Subir .json, .txt/.md o rellenar tabla.
- Publicar -> guarda catalogo.json.
- Probar en simulador.html con mismo tenant.

6. Comandos del cliente en WhatsApp

- hola -> menu numerado
- 1, 2, 3 -> anade servicio
- LISTO -> presupuesto con IVA
- AYUDA / REINICIAR

Los importes vienen de catalogo.json, no del markdown.

7. Formatos de subida

- .json: formato catalogo_ejemplo.json
- .txt/.md: lineas 'Servicio - 450'
- Pegar texto en el panel

8. Endpoints API

```
GET /whatsapp/health
PUT /whatsapp/tenants/{id}/catalogo
POST /whatsapp/tenants/{id}/documentos
POST /whatsapp/tenants/{id}/publicar
POST /whatsapp/presupuesto/simular
GET/POST /whatsapp/webhook
```

9. WhatsApp en produccion

- Meta for Developers -> WhatsApp
- Webhook: `https://TU_DOMINIO/whatsapp/webhook`
- `META_ACCESS_TOKEN`, `META_PHONE_NUMBER_ID`, `META_VERIFY_TOKEN`
- `PRESUPUESTO_TENANT_DEFAULT` = mismo ID del panel

No se usa BuilderBot; API Meta directa en Python.

10. Puertos

- 8020 - bot presupuesto + API
- 8010 - FacturAI
- 8080 - web Openix (aprox.)

11. Esquema en una frase

Subes tarifas en `configuracion.html` -> `data/tenants/.../catalogo.json` -> motor_presupuesto via API 8020 -> simulador o WhatsApp real.

12. Problemas frecuentes

- API: comprobar uvicorn en `127.0.0.1:8020`
- CORS/file://: `python3 -m http.server 8765` en la carpeta
- Menu demo: publicar y usar mismo tenant en simulador

Borrar este PDF cuando ya no lo necesites. Doc viva 🍷 README.md y arquitectura.html.